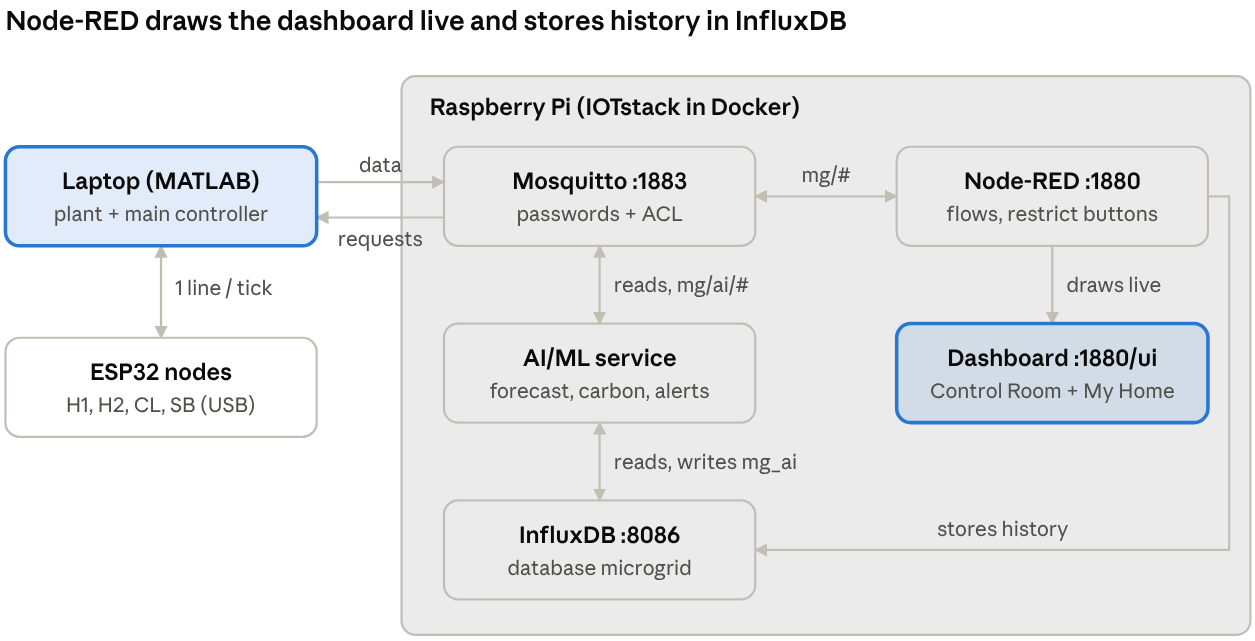
PS #38 Pi IoT Server Setup (Database + Dashboard)

Oct 8, 2026 · @Res

What we are building

The Pi runs three programs in Docker: Mosquitto receives the main controller's messages, and Node-RED both draws the live dashboard (`:1880/ui`) and copies every message into the InfluxDB database `microgrid` for history. The AI/ML service reads the same data and publishes its results on `mg/ai/#` . Nothing on the Pi switches a load: a dashboard button only sends a request, and the main controller on the laptop accepts or refuses it.



data path · laptop, Pi containers, dashboards

1. Install on the Pi (tutorial steps + our changes)

Follow the tutorial (IOTstack: Mosquitto, Node-RED, InfluxDB, Portainer in Docker; we skip the tutorial's Grafana), with three changes for our project.

1. **Stop the old Mosquitto first.** We installed Mosquitto with apt earlier; it holds port 1883 and the IOTstack Mosquitto container will not start. Run `sudo systemctl stop mosquitto` and `sudo systemctl disable mosquitto`.
2. Update and install IOTstack (from the tutorial):
   - `sudo apt update && sudo apt upgrade`
   - `curl -fsSL https://raw.githubusercontent.com/SensorsIot/IOTstack/master/install.sh | bash`
   - `sudo shutdown -r now`
3. `cd ~/IOTstack && ./menu.sh` → select **Mosquitto, Node-RED, InfluxDB, Portainer-CE** → build → Start stack. Check with `docker-compose ps` (newer Pis: `docker compose ps`): all four must say Up.
4. **Create our database** (name below, not the tutorial's `sensor_data`):
   - `docker exec -it influxdb influx`
   - `CREATE DATABASE microgrid`
   - `CREATE RETENTION POLICY "week" ON "microgrid" DURATION 7d REPLICATION 1 DEFAULT`
   - `quit`
5. **Lock down Mosquitto** (section 3), then build the Node-RED flow (section 4) and the dashboard (section 5).
6. Give the Pi a **fixed IP address** on our local network (router DHCP reservation), so the laptop always finds it.

Web pages on the Pi (IOTstack defaults, check on yours): Node-RED editor `http://<pi-ip>:1880`, dashboard `:1880/ui`, InfluxDB `:8086` (no web page in v1.8), Portainer `:9000`, MQTT `:1883`.

2. Database: name, tables and columns

One database called **`microgrid`**, with six measurements (InfluxDB's word for a table). Each MQTT topic from the main controller becomes one measurement. **Tags** are the labels you filter by (which house, which node); **fields** are the numbers you plot. Keep these names exactly: the main controller already sends them (full meaning of every field: `mg_hier/docs/pi_data_contract.md`).

Measurement (table)	Filled from topic	Tags (filter by)	Fields (values)	Dashboard use
`mg_state`	`mg/state` (1 s)	`mode`	`tick, sim_s, grid_available, grid_v, grid_f, grid_p_w, grid_q_var, breaker_closed, bus_v, bus_f, dtheta_deg, soc_pct, bess_p_w, bess_q_var, bess_p_set_w, pv_community_w, pv_houses_w, pv_total_w, load_total_w, price_rs_kwh, ai_status, flags`	Energy flow, status, BESS, PV, grid
`mg_node`	`mg/node/H1`, `H2`, `CL`, `SB` (1 s)	`node`, `link`	`online, voltage_v, current_a, load_p_w, load_q_var, pv_p_w, net_p_w, critical_on, flexible_on, protection_code, comm_lost, import_wh, export_wh, status_code` (SB: `soc_pct, p_w, q_var, p_applied_w, clamped, sync`)	Per-node status, restricted loads
`mg_account`	`mg/house/H1/account`, `H2` (5 s)	`house`	`balance_rs, credit_ok, flexible_allowed, price_rs_kwh, import_wh, export_wh, bought_from_peers_wh, sold_to_peers_wh, status_code`	Users, prepaid, energy sharing; the user dashboard
`mg_event`	`mg/event`	`type`, `node`	`text, sim_time`	Logs
`mg_ack`	`mg/ack/#`	`source`	`id, accepted, reason`	Log of dashboard / AI requests

Measurement (table)	Filled from topic	Tags (filter by)	Fields (values)	Dashboard use
`mg_ai`	written by the AI service (live copy on mg/ai/#)	`kind` (pv_forecast, load_forecast, anomaly, carbon), `model`	`value, horizon_min, unit_text`	Forecasting, predictive analysis, carbon footprint

Rules that avoid broken charts later:

- **Yes/no values are stored as 1/0 numbers** (charts plot numbers; InfluxDB refuses a field that changes type).
- **Text values** (`protection` , `text` , `reason`) are only in `mg_node.protection_code` (0 ok, 1 tripped, 2 locked), `mg_event` and `mg_ack` .
- **Never rename a field**; only add new ones. Messages carry `v` (format version, now 1).
- Carbon footprint = grid import energy × grid emission factor (kg CO_2 per kWh). The AI/ML team picks and cites the factor and writes the result into `mg_ai` .

3. Mosquitto: passwords and who may see what

IOTstack's Mosquitto lets anyone connect by default; we switch that off so only our five users exist and each sees only its own topics.

MQTT user	Who uses it	May read	May send
`mgctl`	Main controller (MATLAB on the laptop)	`mg/cmd/#` (requests)	`mg/state` , `mg/node/#` , `mg/house/#` , `mg/event` , `mg/ack/#`
`dash`	Node-RED (dashboard, database writer, restrict buttons)	`mg/#`	`mg/cmd/op/#`
`ai`	AI/ML service	`mg/#`	`mg/cmd/ai/#` , mg/ai/#
`H1` , `H2`	One per household (user dashboard)	only `mg/house/<own id>/#` , `mg/ack/user/<own id>/#`	only `mg/cmd/user/<own id>/#`

Steps (IOTstack paths; check them on your Pi with `ls ~/IOTstack/volumes/mosquitto`):

1. Copy our generated ACL file `mg_hier/docs/mosquitto/acl` to `~/IOTstack/volumes/mosquitto/config/acl`.
2. In `~/IOTstack/volumes/mosquitto/config/mosquitto.conf` set: `allow_anonymous false`, `password_file /mosquitto/pwfile/pwfile`, `acl_file /mosquitto/config/acl` (paths inside the container).
3. Create each user (repeat for dash, ai, H1, H2): `docker exec mosquitto mosquitto_passwd -b /mosquitto/pwfile/pwfile mgctl <password>`
4. `cd ~/IOTstack && docker-compose restart mosquitto`
5. Three tests, all must behave as written:
   - Anonymous is refused: `docker exec mosquitto mosquitto_sub -h localhost -t 'mg/#'` → "not authorised".
   - H1 sees nothing of H2: subscribe as H1 to `mg/house/H2/#` → no messages while the controller runs.
   - H1 cannot pose as operator: publish as H1 to `mg/cmd/op/restrict` → the main controller never acks it.

Keep the passwords out of the slides and the repo; write them on paper for the demo.

4. Node-RED: MQTT into the database

One flow of three nodes does all of it: **mqtt in → function → influxdb out**.

1. **mqtt in**: server `mosquitto` port `1883` (the container name works inside IOTstack), Security tab user `dash` + its password, topic `mg/#`, QoS 0, output "a parsed JSON object".
2. **function** "to influx": paste the code below.
3. **influxdb out** (palette `node-red-contrib-influxdb`; add it in Manage palette if it is missing): server host `influxdb`, port `8086`, database `microgrid`, **leave Measurement empty** (the function sets it).

```
// mg/# -> InfluxDB measurements (names fixed in section 2)
const p = msg.payload, t = msg.topic.split('/');
const b = x => (x ? 1 : 0);                 // yes/no -> 1/0
if (msg.topic === 'mg/state') {
  global.set('sim_s', p.sim_s);               // simulated time, used by the
restrict buttons
```

```
  msg.measurement = 'mg_state';
  msg.payload = [{
    tick: p.tick, sim_s: p.sim_s,
    grid_available: b(p.grid.available), grid_v: p.grid.voltage_v, grid_f:
p.grid.frequency_hz,
    grid_p_w: p.grid.p_w, grid_q_var: p.grid.q_var, breaker_closed:
b(p.grid.breaker === 'CLOSED'),
    bus_v: p.bus.voltage_v, bus_f: p.bus.frequency_hz, dtheta_deg:
p.bus.dtheta_deg,
    soc_pct: p.bess.soc_pct, bess_p_w: p.bess.p_w, bess_q_var: p.bess.q_var,
bess_p_set_w: p.bess.p_set_w,
    pv_community_w: p.pv.community_w, pv_houses_w: p.pv.houses_w, pv_total_w:
p.pv.total_w,
    load_total_w: p.loads.total_w, price_rs_kwh: p.price_rs_kwh, ai_status:
p.ai_status, flags: p.flags
  }, { mode: p.mode }];
} else if (t[1] === 'node' && t[2] === 'SB') {
  msg.measurement = 'mg_node';
  msg.payload = [{
    online: b(p.online), comm_lost: b(p.node_comm_lost), soc_pct:
p.bess.soc_pct, p_w: p.bess.p_w,
    q_var: p.bess.q_var, p_applied_w: p.inverter.p_applied_w, clamped:
b(p.inverter.clamped),
    sync: b(p.inverter.sync), pv_p_w: p.pv_p_w
  }, { node: 'SB', link: p.link }];
} else if (t[1] === 'node') {
  const m = p.meter, prot = { ok: 0, tripped: 1, locked: 2 };
  msg.measurement = 'mg_node';
  msg.payload = [{
    online: b(p.online), voltage_v: m.voltage_v, current_a: m.current_a,
load_p_w: m.load_p_w,
    load_q_var: m.load_q_var, pv_p_w: m.pv_p_w, net_p_w: m.net_p_w,
    critical_on: b(p.relays.critical_on), flexible_on:
b(p.relays.flexible_on),
    protection_code: prot[p.protection], comm_lost: b(p.node_comm_lost),
    import_wh: p.energy.import_wh, export_wh: p.energy.export_wh,
status_code: p.status_code
  }, { node: p.node_id, link: p.link }];
} else if (t[1] === 'house') {
  msg.measurement = 'mg_account';
  msg.payload = [{
    balance_rs: p.balance_rs, credit_ok: b(p.credit_ok), flexible_allowed:
b(p.flexible_allowed),
```

```
    price_rs_kwh: p.price_rs_kwh, import_wh: p.import_wh, export_wh:
p.export_wh,
    bought_from_peers_wh: p.bought_from_peers_wh, sold_to_peers_wh:
p.sold_to_peers_wh,
    status_code: p.status_code
  }, { house: p.house_id }];
} else if (msg.topic === 'mg/event') {
  msg.measurement = 'mg_event';
  msg.payload = [{ text: p.text, sim_time: p.sim_time }, { type: p.type,
node: p.node }];
} else if (t[1] === 'ack') {
  msg.measurement = 'mg_ack';
  msg.payload = [{ id: p.id, accepted: b(p.accepted), reason: p.reason }, {
source: t.slice(2).join('/') }];
} else {
  return null;                                  // mg/cmd/... requests are not
stored
}
return msg;
```

Test data before the laptop is connected: `mg_hier/results/pi_msgs/*.json` are real example messages; publish one with an inject node or `mosquitto_pub`.

5. Dashboard in Node-RED (node-red-dashboard 3.x)

The dashboard is built in Node-RED with the classic `node-red-dashboard` palette and is fed live from MQTT; InfluxDB keeps the history (logs, AI training). Grafana is not used.

Setup

1. Manage palette → install `node-red-dashboard` and `node-red-node-ui-table` (for the tables). The page is `http://<pi-ip>:1880/ui`.
2. One shared **mqtt-broker** config: server `mosquitto`, port 1883, user `dash`. Every **mqtt in** below uses it with output "a parsed JSON object".
3. Put a login on the page: in `~/IOTstack/volumes/nodered/data/settings.js` set `httpNodeAuth: {user: "team", pass: "<hash>"}` (hash from `docker exec -it nodered node-red admin hash-pw`), then restart Node-RED.

Tabs and widgets (main tab title: *URJA Microgrid – Control Room*)

Tab / group	Widgets	Fed by (mqtt in)
Control Room / Mode and grid	ui_text mode, ui_text breaker, ui_gauge grid V (207–253), ui_gauge frequency (49–51 Hz)	`mg/state`
Control Room / Energy flow	one ui_chart, 4 lines: grid, battery, solar, loads	`mg/state` → function "split power"
Control Room / Battery	ui_gauge SOC %, ui_text power and set-point	`mg/state`
Control Room / Solar	ui_chart community PV and house PV	`mg/state`
Control Room / Nodes	ui_table: H1, H2, CL, SB with online, critical, flexible, protection	`mg/node/+` → function "nodes table"
Control Room / Users and sharing	ui_table: H1, H2 balance, flexible allowed, bought from / sold to neighbours	`mg/house/+/account`
Control Room / Restrict loads	ui_button "Restrict CL", "Restrict H2", "Restrict H1", "Release all"; ui_text last answer	sends `mg/cmd/op/restrict`; reads `mg/ack/op`
Control Room / Event log	ui_template, last 20 events	`mg/event`
Control Room / AI	ui_chart forecast vs actual, ui_text carbon today, ui_text anomalies	`mg/ai/#` (published by the AI service)
My Home H1, My Home H2	balance, flexible on/off, import/export, bought/sold, own voltage and load. **No AI.**	`mg/house/H1/account`, `mg/node/H1` (H2 alike)

Function "split power" (one chart, four lines; the chart names each line by `msg.topic`):

```
const p = msg.payload;
return [[
  { topic: 'Grid import (W)', payload: p.grid.p_w },
  { topic: 'Battery (W)',     payload: p.bess.p_w },
  { topic: 'Solar (W)',       payload: p.pv.total_w },
  { topic: 'Loads (W)',       payload: p.loads.total_w }
]];
```

Function "nodes table" (keeps the latest row per node):

```
const rows = context.get('rows') || {};
const p = msg.payload;
rows[p.node_id] = (p.node_id === 'SB')
  ? { node: 'SB', online: p.online, info: `SOC ${p.bess.soc_pct.toFixed(1)}
%, ${Math.round(p.bess.p_w)} W` }
  : { node: p.node_id, online: p.online, critical: p.relays.critical_on,
      flexible: p.relays.flexible_on, protection: p.protection };
context.set('rows', rows);
msg.payload = Object.values(rows);
return msg;
```

Restrict buttons: each ui_button sends its target as payload (`CL` , `H2` , `H1`) → function below → **mqtt out** (broker config `dash` , topic set by the function). "Release all" sends the same with `value: 0` for each target.

```
msg.topic = 'mg/cmd/op/restrict';
msg.payload = { id: Date.now(), t_sent: global.get('sim_s'),   // simulated
time, from mg/state
                target: msg.payload, value: 1, ttl: 600 };     // hold its
flexible load off 10 min
return msg;
```

`t_sent` must be the **simulated** time (set by the database flow in section 4): the main controller refuses a request more than 10 s old. Its answer (accepted or the reason it was refused) arrives on `mg/ack/op` .

Event log: `mg/event` → function below → ui_template with `<div ng-repeat="l in msg.payload">{{l}}</div>` .

```
const line = msg.payload.sim_time + '  ' + msg.payload.node + ': ' +
msg.payload.text;
const log = [line].concat(context.get('log') || []).slice(0, 20);   // newest
first, last 20
context.set('log', log);
msg.payload = log;
return msg;
```

Privacy limit: the classic dashboard has one login for all tabs, so whoever can open `/ui` can open both My Home tabs. For the demo show each My Home tab on its own screen and say this plainly. Real per-house privacy needs a separate page per house logged in to MQTT as `H1` (the broker then delivers only H1's data). This package is also no longer maintained; fine for the hackathon.

6. Checks, pitfalls, and what is not ready yet

Done when all of these hold:

- [ ] `docker-compose ps` shows 4 containers Up after a Pi reboot
- [ ] Anonymous MQTT refused; H1 cannot read H2 (section 3 tests)
- [ ] `docker exec -it influxdb influx` → `USE microgrid` → `SHOW MEASUREMENTS` lists `mg_state, mg_node, mg_account, mg_event`
- [ ] `/ui` asks for the login, and the Energy flow chart moves while the controller runs
- [ ] "Restrict CL" turns the pump's flexible relay off within 2 s and shows "accepted"

Pitfalls:

- **Port 1883 busy** → the old apt Mosquitto is still running (section 1, step 1).
- **"field type conflict" in Node-RED** → a field was once written as text or true/false. Fix the function, then in influx: `DROP MEASUREMENT mg_node` (that table only) and let it refill.
- **Dashboard chart stays empty** → time range too short or the Pi clock is wrong (`timedatectl`); data are stamped with the Pi's clock on arrival.
- **Simulation faster than real time** → the chart's time axis squeezes; for the demo run the controller at 1 tick per second (`pace_s = 1`). `sim_time` is stored for the logs.
- **SD card wear** → 7-day retention (section 1) keeps it small.

Not ready yet (our side, not the dashboard team's): the main controller publishes to the **simulated** broker only; the real MATLAB → Pi MQTT link (user `mgctl`) is the next step, and the example JSON files appear after the next MATLAB run. Not checked on a real Pi: the IOTstack file paths and default ports above.